

Prog 4 – ZH-2

Készítse el a megadott projektben az alábbi feladatokat. Használja ehhez a projektben található service osztályokat. A feladat megkezdése előtt alaposan olvassa át a teljesen feladatsort! A feladathoz szüksége van egy adatbázisra. Amennyiben elindítja a projektben található compose-t, akkor elindul az adatbázis és minden szükséges objektum létrejön benne.

Az adatbáziskapcsolathoz szükséges resource definíciója:

```
<Resource name="jdbc/MariaDB" type="javax.sql.DataSource"
driverClassName="org.mariadb.jdbc.Driver"
factory="org.apache.tomcat.jdbc.pool.DataSourceFactory"
url="jdbc:mariadb://localhost:3306/zh2_db" username="mariadb" password="Password111"
maxActive="20" maxIdle="10" maxWait="-1" />
```

Az adatbázisban két felhasználó van.

user1: neki user jogosultsága van

user2: neki user és creator jogosultsága van

Mind a kettőnek "Password111" a jelszava. A jelszavak Bcrypttel vannak elforgatva.

Adatbázis

1. Implementálja a következő adat elérési osztályokat és a bennük lévő metódusokat:

Repository, CarRepository, RoleRepository, UserRepository

Az implementáláshoz használja az osztályokban lévő SQL lekérdezéseket.

SOAP

2. A projektben található CarDataService és SoapCarDataService osztályok alapján generáljon WSDL-t az src/main/resources/wsdl mappába. (cxfr-java2ws-plugin)

3. A kigenerált WSDL-eket felhasználva generáljon egy klienst ehhez a web service-hez. Ezeket az osztályokat közvetlenül a target/generated mappába generátassa ki. (jaxws-maven-plugin)

4. A hu.pte.mik.prog4.zh2.service.CarService osztálynak implementálja a getRemoteKm metódusát. Ez hívja meg a korábban kigenerált WS klienst.

REST

5. Implementálja a CarController osztályt mint REST controller osztály. Várjon JSON-öket befelé jövet és JSON-öket is adjon kifelé is. Hozzon benne létre egy methodot, ami meghívja a korábban implementált service hívást és visszaadja a km értéket. Figyeljen, hogy milyen path-t használ!!

6. A carList.jsp oldalon lévő KM gombot implementálja. Hívja meg JavaScript segítségével a fentebb implementált REST endpointot. Akár sikeres volt a hívás, akár nem, a böngésző dobjon róla egy értesítést.

localhost:8080 says

The sum KM: 555627

localhost:8080 says

Some error occurred!

OK

Security

7. A login.html állományba készítse el a bejelentkező formot.

8. Implementálja az AuthenticationModule osztálynak a metódusait úgy, hogy az adatbázisból authenticáljon a rendszer. Ehhez használja a UserRepository és a RoleRepository osztályokat.

9. Konfigurálja be az authenticálást a különböző állományokban a következő képpen:

Legyen FORM alapú bejelentkezés.

Az index.jsp-t el lehessen érni bejelentkezés nélkül.

A soap-os webservice-t is el lehessen érni bejelentkezés nélkül. (Erre azért van szükség, mert azt kódból hívjuk és nem foglalkoztunk az ottani authenticálással.)

Az összes többi oldal eléréséhez be kell jelentkezni és a “user” role-hoz legyen kötve.

Az új autó létrehozása a “creator” role-hoz legyen kötve.

A bejelentkező oldal legyen a login.html.

Hibás bejelentkezés esetén irányítson át a login_failed.html-re.

10. Implementálja a kijelentkezést is a /logout pathra. Ha sikeres volt a kijelentkezés irányítsa át a rendszer a felhasználót a context path-re.

11. Készítsen egy felületet, ahol lehet regisztrálni. Ezt az oldalt bejelentkezés nélkül is el lehessen érni. A regisztrációkor meg kell adni egy felhasználó nevet és egy jelszót. A felhasználónevet ellenőrizzük, hogy egyedi legyen. A jelszót forgassuk el Bcrypttel. A frissen regisztrált felhasználó kappjon user role-t. Regisztráció után a felhasználót irányítsuk át a bejelentkező oldalra.